Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas
Universidad Complutense de Madrid
Óscar Martín — Curso 2026/27

25. Entrada y salida

El problema

- La programación declarativa no permite efectos laterales.
  - Pero leer y escribir en el terminal son efectos laterales.
- Transparencia referencial:
  - `getChar` da un resultado distinto cada vez que se llama.
  - ¿`leeInt - leeInt = 0`?

La solución: `IO`

- Tomemos el estado del mundo como un argumento: `ElMundo -> (a, ElMundo)`
  - `getChar :: ElMundo -> (Char, ElMundo)`.
- Esto es (más o menos) lo que hace `IO a`.
- `IO` es un constructor de tipos.
  - `IO` está implementado a bajo nivel.
  - `ElMundo` no existe.
- Los valores del tipo `IO a` se llaman *acciones*.
  - `getChar :: IO Char`
- Hay acciones que no producen un resultado: `IO ()`.
- `() :: ()` (*unidad*)
  - Por cierto, también existe `Void`.
- Hay acciones que necesitan un argumento: `putChar :: Char -> IO ()`
- Los objetos de tipo `IO a` son objetos normales de Haskell.

```haskell
acciones :: [IO ()]
acciones = [putChar 'a', putChar 'x', putChar 'z']
```

  - No las ejecuta, solo las deja ahí almacenadas.
  - Igualmente, `letra = getChar` no ejecuta `getChar`, solo le da otro nombre.
- Las acciones de un objeto `IO a` se ejecutan cuando:
  - se escriben en `ghci`,
  - forman parte (transitivamente) del `main` de un programa.
- Intuición:
  - Una expresión de tipo `IO a` *encapsula* una acción en el mundo real…
    - a resultas de la cual se obtiene un valor de tipo `a`.
  - Una expresión de tipo `IO a` es una receta: "¿qué habría que hacer para…?"
    - Una receta no tiene efectos: su ejecución los tiene.
    - `getChar :: IO Char` es siempre la misma receta.
- Se preserva la transparencia referencial:
  - `leeInt :: IO Int`, luego `leeInt - leeInt` no está bien tipado.

Algunas funciones básicas de `IO`

- `getChar :: IO Char`
- `getLine :: IO String`
  - error: `saludo = "Hola, me llamo " ++ getLine`
- `putChar :: Char -> IO ()`
- `putStr :: String -> IO ()`
- `putStrLn :: String -> IO ()`
- `print :: Show a => a -> IO ()`
  - `print = putStrLn . show`
  - `print (3 * 5)`
  - `print "¡Caramba!"`
- En `ghci`, `print` se llama automáticamente cuando escribimos una expresión.

`>>=`

- ¿Cómo podemos ejecutar `putStrLn` tras `getLine`?

```
infixl 1 >>=
(>>=) :: IO a -> (a -> IO b) -> IO b
```

```
eco :: IO ()
eco = getLine >>= putStrLn
```

- `>>=` extrae el valor de tipo `a` del primer argumento y lo pasa al segundo.

- La definición de `>>=` para `IO` es interna en la implementación de Haskell.
- No siempre el encadenamiento directo es posible.
  - Recuerda que `f` es lo mismo que `\x -> f x`.
  - `eco = getLine >>= \l -> putStrLn l`

```
ecoeco :: IO ()
ecoeco = getLine
         >>= \l -> putStrLn (l ++ " " ++ l)
```

```
tomaNombre :: IO String
tomaNombre = putStrLn "¿Cuál es tu nombre?"
             >>= \_ -> getLine
```

`do` y `<-`

- En vez de esto:

```
quien :: IO ()
quien = putStrLn "¿Cuál es tu nombre?"
        >>= \_ -> getLine
        >>= \nm -> putStrLn "¿Y tu apellido?"
        >>= \_ -> getLine
        >>= \ap -> putStrLn ("Hola, " ++ nm ++ " " ++ ap ++ ".")
```

- … podemos escribir esto:

```
quien' :: IO ()
quien' = do putStrLn "¿Cuál es tu nombre?"
            nm <- getLine
            putStrLn "¿Y tu apellido?"
            ap <- getLine
            putStrLn ("Hola, " ++ nm ++ " " ++ ap ++ ".")
```

- El tipo y el valor de un bloque `do` es el de su última acción.
- El encolumnado es importante.
- (El cuerpo de una λ-expresión se extiende a la derecha tanto como sea posible.)

`return`

- ¿Y si necesito tener un valor de tipo `IO a` a partir de un valor de tipo `a`?
  - Por ejemplo: necesito una acción `IO String` cuyo resultado sea `"Hola"`.

```
return :: a -> IO a
```

- `return` es una función que construye valores de tipo `IO a`.
- `return x` es la acción que no hace nada y produce `x` como resultado.

```haskell
dobleNombre :: IO String
dobleNombre = do putStrLn "¿Cuál es tu nombre?"
                 nombre <- getLine
                 return (nombre ++ " - " ++ nombre)
```

- En algún sentido, `return` y `<-` son opuestos.
  - `x <- return 42` pone el valor `42` en `x`
- No te dejes confundir por el nombre: `return` no termina la ejecución.

Más ejemplos

```haskell
putStrLn :: String -> IO ()
putStrLn s = do putStr s
                putChar '\n'
```

```haskell
unoYTres :: IO (Char, Char)
unoYTres = do x <- getChar
              getChar
              y <- getChar
              return (x, y)

dosYCuatroJuntos :: IO String
dosYCuatroJuntos = do getChar
                      (x, y) <- unoYTres
                      return [x, y]
```

- Lo de la izquierda de `<-` es un patrón.

```haskell
leeInt :: IO Int
leeInt = do linea <- getLine
            return (read linea)

leeYSumaDosNums :: IO Int
leeYSumaDosNums = do
    int1 <- leeInt
    int2 <- leeInt
    return (int1 + int2)

leeDosNumsYEscribeSuma :: IO ()
leeDosNumsYEscribeSuma = leeYSumaDosNums >>= print
```

`let`

```haskell
leerYMostrarLongitud :: IO ()
leerYMostrarLongitud = do
    linea <- getLine
```

```
    let n = length linea
    putStrLn ("La línea tiene " ++ show n ++ " caracteres.")
```

- Estos `let` no tienen `in`.
- A la derecha del `=` puede aparecer cualquier código *puro* (sin acciones).
- También tiene una traducción (sencilla) a `>>=`.

`if`, `case`, ...

- Dentro de un bloque `do`, cada expresión debe tener el tipo `IO a` apropiado.
- `if` y `case` son válidos.
- También son válidas las llamadas a funciones.
- También `do` anidados.
  - `do` puede ir en cualquier sitio donde se requiera devolver un valor `IO a`.

```
dosOportunidades :: IO ()
dosOportunidades = do
    putStr "Escribe un número par: "
    n <- leeInt
    if even n then
        putStrLn "Bien hecho."
    else do putStr "Mal. Te doy otra oportunidad: "
            n <- leeInt
            if even n then putStrLn "Bien ahora."
                      else putStrLn "Mal de nuevo."
```

- Por cierto: una `n` oculta a la otra, lo que en este caso parece aceptable.

Recursión

```
putStr :: String -> IO ()
putStr [] = return ()
putStr (x:xs) = do putChar x
                   putStr xs
```

```
getLine :: IO String
getLine = do x <- getChar
             if x == '\n' then
                 return []
             else
                 do xs <- getLine
                    return (x:xs)
```

Ejercicios

1. ¿Qué tipo tiene cada una de estas expresiones? Si no están bien tipadas, explica por qué.
   a) `putStrLn "Hola" ++ "!"`
   b) `length getLine`
   c) `getLine >>= \l -> return (length l)`
   d) `getLine >>= \l -> length l`
   e) `[getLine, return "hola"]`
   f) La expresión
   ```
   do n <- leeInt
      putStrLn n
   ```

2. Define `leeIntEnRango :: Int -> Int -> IO Int`, que pide un número entre dos límites y lo vuelve a pedir hasta que esté dentro de ellos:
   ```
   Introduce un número entre 1 y 10:
   15
   Mal, repite.
   Introduce un número entre 1 y 10:
   7
   ```

3. Define `lineasAlReves :: IO ()`, que lee líneas hasta que lee una vacía y las escribe en orden inverso.

4. Las acciones son valores: se pueden guardar en listas y pasar a funciones.
   i) Define por recursión `ejecuta :: [IO ()] -> IO ()`, que ejecuta en orden las acciones de la lista: `ejecuta [putChar 'a', putChar 'x', putChar 'z']` escribe `axz`.
   ii) Define `ejecuta` con `foldr`.
   iii) Define `secuencia :: [IO a] -> IO [a]`, que además devuelve la lista de los resultados.
   iv) Usa `secuencia` para definir `leeLineas :: Int -> IO [String]`, que lee el número de líneas indicado.

   (Las funciones `ejecuta` y `secuencia` están definidas en el preludio como `sequence_` y `sequence`.)